

Teoria algorytmów i obliczeń - laboratorium

Zadanie laboratoryjne - multigrafy

Amadeusz Nowak

Andrii Voznesenskyi

Oleh Kiprik

Piotr Padamczyk

31 grudnia 2024

Spis treści

1	Definicje	1
2	Przykłady działania programu	3
3	Opis algorytmów	6
3.1	Algorytm obliczania rozmiaru multigrafu	6
3.2	Algorytm wyszukiwania cykli w multigrafie	6
3.3	Algorytm liczenia cykli Hamiltona	7
3.4	Algorytm obliczania metryki pomiędzy dwoma multigrafami	8
3.5	Algorytm minimalnego rozszerzenia dla cyklu Hamiltona	8
4	Dowód poprawności algorytmów	9
4.1	Dowód poprawności algorytmu obliczania rozmiaru multigrafu	9
4.2	Dowód poprawności algorytmu wyszukiwania cykli w multigrafie	10
4.3	Dowód poprawności algorytmu liczenia cykli Hamiltona	10
4.4	Dowód poprawności algorytmu obliczania metryki pomiędzy dwoma multigrafami	11
4.5	Dowód poprawności algorytmu minimalnego rozszerzenia dla cyklu Hamiltona	12
5	Instrukcja obsługi	14
5.1	Struktura projektu	14
5.2	Kompilacja projektu	14
5.3	Uruchamianie programu głównego	14
5.4	Opis funkcji programu głównego	15
5.5	Uruchamianie generatora grafów	15
5.6	Przykłady użycia generatora grafów	16
5.7	Uruchamianie testów	16
6	Wnioski	16

1 Definicje

Definicja 1: Multigraf.

Multigraf to uporządkowana para $G = (V, E)$, gdzie:

- V jest skończonym zbiorem wierzchołków,
- E jest multizbiorem krawędzi, z których każda łączy parę wierzchołków z V . Krawędzie mogą się powtarzać, co oznacza możliwość występowania wielokrotnych krawędzi między tą samą parą wierzchołków. Nie rozważamy krawędzi w postaci pętli tj. krawędzie (v, v) .

Definicja 2: Rozmiar multigrafu.

Rozmiar multigrafu $G = (V, E)$, oznaczany $|E|$, to całkowita liczba krawędzi w multigrafie, uwzględniająca wszystkie wielokrotne krawędzie. Formalnie:

$$|E| = \sum_{(u,v) \in V \times V} \text{Liczność}((u, v), E),$$

gdzie $\text{Liczność}((u, v), E)$ to liczba wystąpień krawędzi (u, v) w E .

Definicja 3: Cykl w multigrafie.

Cykl w multigrafie $G = (V, E)$ to ciąg wierzchołków $v_1, v_2, \dots, v_k, v_1$, spełniający następujące warunki:

- $v_i \in V$ dla $i = 1, 2, \dots, k$,
- $(v_i, v_{i+1}) \in E$ dla $i = 1, 2, \dots, k-1$, oraz $(v_k, v_1) \in E$,
- $v_i \neq v_j$ dla $1 \leq i < j \leq k$, z wyjątkiem $v_1 = v_k$.

Definicja 4: Maksymalny cykl w multigrafie.

Maksymalny cykl w multigrafie G to cykl, który zawiera największą liczbę wierzchołków spośród wszystkich cykli w G .

Definicja 5: Cykl Hamiltona.

Cykl Hamiltona w multigrafie $G = (V, E)$ to cykl, który odwiedza każdy wierzchołek z V dokładnie raz i powraca do wierzchołka początkowego. Formalnie, jest to ciąg $v_1, v_2, \dots, v_{|V|}, v_1$, taki że:

- $v_i \in V$ dla $i = 1, 2, \dots, |V|$,
- $(v_i, v_{i+1}) \in E$ dla $i = 1, 2, \dots, |V| - 1$, oraz $(v_{|V|}, v_1) \in E$,
- $v_i \neq v_j$ dla $1 \leq i < j \leq |V|$.

Definicja 6: Minimalne rozszerzenie multigrafu.

Minimalne rozszerzenie multigrafu $G = (V, E)$ to najmniejsza liczba krawędzi, które należy dodać do G , aby istniał w nim cykl Hamiltona.

Definicja 7: Metryka w przestrzeni multigrafów.

Metryką w przestrzeni multigrafów nazwiemy minimalną liczbą operacji potrzebnych do przekształcenia jednego grafu w drugi. Do operacji zaliczamy:

- dodanie krawędzi (również wielokrotnie),
- usunięcie krawędzi,
- dodanie wierzchołka,
- usunięcie wierzchołka,
- zamianę wierzchołków.

$$d(G, H) = \min_{\text{wszystkie sekwencje edycji}} \sum_{\text{operacje edycji grafu}} f_{\text{cost}}(\text{operacja}),$$

Przyjmujemy koszt wszystkich operacji edycji jako 1, oprócz zamiany wierzchołków, która ma koszt 0.

2 Przykłady działania programu

Przykład 1: Multigraf bez pętli i z cyklem Hamiltona

Argumenty wejściowe:

- Liczba grafów: 1
- Liczba wierzchołków w grafie: 3
- Macierz sąsiedztwa:

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

gdzie $A[i][j]$ oznacza liczbę krawędzi pomiędzy wierzchołkiem i a j .

Wynik:

- Rozmiar grafu (liczba krawędzi): 4
- Wszystkie cykle: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$
- Cykl Hamiltona: Tak ($0 \rightarrow 1 \rightarrow 2 \rightarrow 0$)
- Minimalne rozszerzenie dla cyklu Hamiltona: 0
- Maksymalna długość cyklu: 3

Przykład 2: Multigraf z pętlami i cyklami różnej długości

Argumenty wejściowe:

- Liczba grafów: 2
- Graf 1: 3 wierzchołki, macierz sąsiedztwa:

$$A_1 = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

- Graf 2: 4 wierzchołki, macierz sąsiedztwa:

$$A_2 = \begin{bmatrix} 0 & 2 & 0 & 1 \\ 2 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Wynik:

- **Graf 1:**
 - Rozmiar grafu: 2
 - Wszystkie cykle: Brak
 - Cykl Hamiltona: Nie
 - Minimalne rozszerzenie dla cyklu Hamiltona: 1
 - Maksymalna długość cyklu: 2
- **Graf 2:**

- Rozmiar grafu: 5
 - Wszystkie cykle: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$
 - Cykl Hamiltona: Tak ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$)
 - Minimalne rozszerzenie dla cyklu Hamiltona: 0
 - Maksymalna długość cyklu: 4
- Metryka podobieństwa pomiędzy grafem 1 i grafem 2: 1.167

Przykład 3: Minimalne rozszerzenie dla cyklu Hamiltona

Argumenty wejściowe:

- Liczba grafów: 3
- Graf 1: 3 wierzchołki, macierz sąsiedztwa:

$$A_1 = \begin{bmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

- Graf 2: 4 wierzchołki, macierz sąsiedztwa:

$$A_2 = \begin{bmatrix} 0 & 3 & 1 & 0 \\ 3 & 0 & 0 & 2 \\ 1 & 0 & 0 & 1 \\ 0 & 2 & 1 & 0 \end{bmatrix}$$

- Graf 3: 4 wierzchołki, macierz sąsiedztwa:

$$A_3 = \begin{bmatrix} 0 & 2 & 0 & 0 \\ 2 & 0 & 1 & 2 \\ 0 & 1 & 0 & 1 \\ 0 & 2 & 1 & 0 \end{bmatrix}$$

Wynik:

- **Graf 1:**
 - Rozmiar grafu: 4
 - Wszystkie cykle: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$
 - Cykl Hamiltona: Tak ($0 \rightarrow 1 \rightarrow 2 \rightarrow 0$)
 - Minimalne rozszerzenie dla cyklu Hamiltona: 0
 - Maksymalna długość cyklu: 3
- **Graf 2:**
 - Rozmiar grafu: 7
 - Wszystkie cykle: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$
 - Cykl Hamiltona: Tak ($0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$)
 - Minimalne rozszerzenie dla cyklu Hamiltona: 0
 - Maksymalna długość cyklu: 4
- **Graf 3:**

- Rozmiar grafu: 6
- Wszystkie cykle: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$
- Cykl Hamiltona: Nie
- Minimalne rozszerzenie dla cyklu Hamiltona: 1
- Maksymalna długość cyklu: 3
- Metryka podobieństwa pomiędzy grafem 1 i grafem 2: 0.833
- Metryka podobieństwa pomiędzy grafem 2 i grafem 3: 0.500

Przykład 4: Metryka w przestrzeni multigrafów

Argumenty wejściowe:

- Liczba grafów: 3
- Graf 1: 3 wierzchołki, macierz sąsiedztwa:

$$A_1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- Graf 2: 2 wierzchołki, macierz sąsiedztwa:

$$A_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- Graf 3: 5 wierzchołków, macierz sąsiedztwa:

$$A_3 = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 2 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 2 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Wynik:

- **Graf 1:**
 - Rozmiar grafu: 0
 - Wszystkie cykle: Brak
 - Cykl Hamiltona: Nie
 - Minimalne rozszerzenie dla cyklu Hamiltona: 3
 - Maksymalna długość cyklu: 0
- **Graf 2:**
 - Rozmiar grafu: 1
 - Wszystkie cykle: Brak
 - Cykl Hamiltona: Tak ($0 \rightarrow 1 \rightarrow 0$)
 - Minimalne rozszerzenie dla cyklu Hamiltona: 0
 - Maksymalna długość cyklu: 2
- **Graf 3:**

- Rozmiar grafu: 8
 - Wszystkie cykle: 6
 - Cykl Hamiltona: Tak ($0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 0$)
 - Minimalne rozszerzenie dla cyklu Hamiltona: 0
 - Maksymalna długość cyklu: 5
- Metryka podobieństwa pomiędzy grafem 1 i grafem 2: 1.000
 - Metryka podobieństwa pomiędzy grafem 2 i grafem 3: 2.200

3 Opis algorytmów

3.1 Algorytm obliczania rozmiaru multigrafu

Algorithm 1 Obliczanie rozmiaru multigrafu

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A

Ensure: Rozmiar multigrafu $|E|$

```

1:  $|E| \leftarrow 0$ 
2: for each wierzchołki  $i, j$  w  $V$ 
3:    $|E| \leftarrow |E| + A[i][j]$ 
4: end for
5: return  $|E|$ 

```

Złożoność obliczeniowa algorytmu wynosi $O(n^2)$, gdzie n to liczba wierzchołków w multigrafie. Wynika to z iteracji po całej macierzy sąsiedztwa A o wymiarach $n \times n$.

3.2 Algorytm wyszukiwania cykli w multigrafie

Algorithm 2 Znajdowanie cykli w multigrafie

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A

Ensure: Liczba cykli w G

```

1: cycle_count  $\leftarrow 0$ 
2: Utwórz pusty zbiór visited
3: function DFS(( u, start, path ))
4:   if length(path) > 2 and  $u = \text{start}$  then
5:     cycle_count  $\leftarrow$  cycle_count + 1
6:     return
7:   end if
8:   for each  $v$  sąsiadujący z  $u$ 
9:     if  $v \notin \text{visited}$  then
10:      Dodaj  $v$  do visited
11:      DFS(( v, start, path + [v] ))
12:      Usuń  $v$  z visited
13:     end if
14:   end for
15: end function
16: for each wierzchołek  $u \in V$ 
17:   visited  $\leftarrow \{u\}$ 
18:   DFS(( u, u, [u] ))
19: end for
20: return cycle_count

```

Złożoność obliczeniowa algorytmu DFS wynosi $O(V + E)$. W przypadku multigrafu, gdzie V może być większe niż E , złożoność obliczeniowa wynosi $O(V^2)$. Ponieważ algorytm DFS jest wywoływany dla każdego wierzchołka, złożoność obliczeniowa całego algorytmu wynosi $O(V(V + E))$ lub $O(V^3)$ dla przytoczonego drugiego przypadku.

3.3 Algorytm liczenia cykli Hamiltona

Algorithm 3 Liczenie cykli Hamiltona w multigrafie

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A

Ensure: Liczba cykli Hamiltona

```

1: count  $\leftarrow$  0
2: function HAMILTONIANBACKTRACK(( u, visited, depth ))
3:   if depth =  $|V|$  and  $A[u][start] > 0$  then
4:     count  $\leftarrow$  count + 1
5:     return
6:   end if
7:   for each  $v$  sąsiadujący z  $u$ 
8:     if  $v \notin$  visited then
9:       Dodaj  $v$  do visited
10:      HAMILTONIANBACKTRACK(( v, visited, depth + 1 ))
11:      Usuń  $v$  z visited
12:    end if
13:  end for
14: end function
15: for each wierzchołek  $u \in V$ 
16:   visited  $\leftarrow \{u\}$ 
17:   HAMILTONIANBACKTRACK(( u, visited, 1 ))
18: end for
19: return count

```

Złożoność obliczeniowa algorytmu liczenia cykli Hamiltona wynosi $O(V!)$. Wynika to z wykonywania funkcji rekurencyjnej do znajdowania wszystkich ścieżek. Z każdym poziomem głębokości funkcji rekurencyjnej mamy o jeden mniej wierzchołek do wyboru, co daje $V!$ możliwości.

3.4 Algorytm obliczania metryki pomiędzy dwoma multigrafami

Algorithm 4 Obliczanie metryki pomiędzy dwoma multigrafami

Require: $G = (V_G, E_G)$ oraz $H = (V_H, E_H)$ - dwa multigrafy reprezentowane przez macierze sąsiedztwa A_G i A_H oraz $|V_G| \geq |V_H|$.

Ensure: Metryka pomiędzy G i H

```
1: metric  $\leftarrow \infty$ 
2: for each permutacja  $p$  wierzchołków  $V_G$ 
3:   tmp  $\leftarrow 0$ 
4:   for each wierzchołki  $i, j$  w  $V_H$ 
5:     if  $i \geq V_H$  OR  $j \geq V_H$  then
6:       tmp  $\leftarrow$  tmp +  $|A_G[p[i]][p[j]]|$ 
7:     else if then
8:       tmp  $\leftarrow$  tmp +  $|A_G[p[i]][p[j]] - A_H[i][j]|$ 
9:     end if
10:  end for
11: end for
12: metric  $\leftarrow$  metric +  $|V_G| - |V_H|$ 
13: return metric
```

Złożoność algorytmu obliczania metryki dla grafów $G = (V_G, E_G)$ oraz $H = (V_H, E_H)$ wynosi $O(V_G!)$, gdzie graf G ma większą liczbę wierzchołków niż H .

3.5 Algorytm minimalnego rozszerzenia dla cyklu Hamiltona

Algorithm 5 Minimalne rozszerzenie dla cyklu Hamiltona

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A

Ensure: Minimalna liczba krawędzi do dodania, aby uzyskać cykl Hamiltona

```
1: min_edges  $\leftarrow \infty$ 
2: function HASHAMILTONIANCYCLE( $G, V$ )
3:   cycle_count  $\leftarrow$  COUNTHAMILTONIANCYCLES( $G, V$ )
4:   return cycle_count > 0
5: end function
6: function EXTENDGRAPH( $G, \text{added\_edges}$ )
7:   if added_edges  $\geq$  min_edges then
8:     return
9:   end if
10:  if HASHAMILTONIANCYCLE( $G, V$ ) then
11:    min_edges  $\leftarrow$  added_edges
12:    return
13:  end if
14:  for each  $u, v \in V$ 
15:    if  $u \neq v$  i  $A[u][v] = 0$  then
16:      Dodaj krawędź  $(u, v)$  do  $A$ 
17:      EXTENDGRAPH( $G, \text{added\_edges} + 1$ )
18:      Usuń krawędź  $(u, v)$  z  $A$ 
19:    end if
20:  end for
21: end function
22: EXTENDGRAPH( $G, 0$ )
23: return min_edges
```

Złożoność algorytmu wynosi $O(2^m \cdot n!)$, gdzie n to liczba wierzchołków, a $m = \binom{n}{2} - |E|$. Wynika to z tego, że tworzymy grafy poprzez dodanie jeszcze nie istniejących krawędzi, i dla każdego takiego grafu wykonujemy algorytm 3.

4 Dowód poprawności algorytmów

4.1 Dowód poprawności algorytmu obliczania rozmiaru multigrafu

Aby formalnie udowodnić poprawność algorytmu OBLICZROZMIARMULTIGRAFU (Algorytm 1), wykorzystamy indukcję matematyczną.

Teza: Dla dowolnej macierzy sąsiedztwa A grafu $G = (V, E)$, algorytm poprawnie oblicza liczbę krawędzi $|E| = \sum_{i=1}^n \sum_{j=1}^n A[i][j]$, gdzie $n = |V|$.

Podstawa indukcji: Rozważmy graf o $n = 1$ wierzchołku. Macierz sąsiedztwa A ma wymiar 1×1 i zawiera jedną wartość $A[1][1]$, która reprezentuje liczbę pętli w grafie.

Dla $n = 1$, algorytm: 1. Inicjalizuje $|E| \leftarrow 0$. 2. Iteruje po macierzy A , która zawiera jedynie $A[1][1]$. 3. Dodaje $A[1][1]$ do $|E|$.

Po zakończeniu iteracji algorytm zwraca $|E| = A[1][1]$, co jest zgodne z definicją liczby krawędzi grafu. Podstawa indukcji została zatem wykazana.

Krok indukcyjny: Załóżmy, że algorytm poprawnie oblicza liczbę krawędzi dla dowolnego grafu o $n = k$ wierzchołkach. Udowodnijmy, że algorytm działa poprawnie dla $n = k + 1$ wierzchołków.

Założenie indukcyjne: Dla grafu $G_k = (V_k, E_k)$ o k wierzchołkach macierz sąsiedztwa A_k jest rozmiaru $k \times k$, a algorytm zwraca poprawną wartość:

$$|E_k| = \sum_{i=1}^k \sum_{j=1}^k A_k[i][j].$$

Dowód dla $n = k + 1$: Rozważmy graf $G_{k+1} = (V_{k+1}, E_{k+1})$ z macierzą sąsiedztwa A_{k+1} , która jest rozmiaru $(k + 1) \times (k + 1)$. Macierz A_{k+1} można przedstawić jako:

$$A_{k+1} = \begin{bmatrix} A_k & b \\ c^\top & d \end{bmatrix},$$

gdzie: - A_k jest podmacierzą $k \times k$ dla grafu G_k , - b i $c^\top$ to wektory krawędzi pomiędzy nowym wierzchołkiem a istniejącymi k wierzchołkami, - d to liczba pętli w nowym wierzchołku.

Algorytm iteruje po wszystkich $(k + 1)^2$ elementach macierzy A_{k+1} . W trakcie iteracji: 1. Sumuje wszystkie elementy $A_k[i][j]$ z podmacierzy A_k , zgodnie z założeniem indukcyjnym. 2. Dodaje wartości z wektorów b , $c^\top$, oraz d .

Łączna suma, jaką zwraca algorytm, wynosi:

$$|E_{k+1}| = \sum_{i=1}^k \sum_{j=1}^k A_k[i][j] + \sum_{i=1}^k b[i] + \sum_{j=1}^k c[j] + d.$$

Jest to zgodne z definicją liczby krawędzi w grafie o $k + 1$ wierzchołkach:

$$|E_{k+1}| = \sum_{i=1}^{k+1} \sum_{j=1}^{k+1} A_{k+1}[i][j].$$

Wniosek: Na mocy zasady indukcji matematycznej algorytm OBLICZROZMIARMULTIGRAFU działa poprawnie dla dowolnej macierzy sąsiedztwa A grafu G .

4.2 Dowód poprawności algorytmu wyszukiwania cykli w multigrafie

Aby udowodnić poprawność algorytmu ZNAJDŹCYKLE (Algorytm 2), wykorzystamy indukcję matematyczną w oparciu o liczbę wierzchołków $n = |V|$.

Teza: Dla grafu $G = (V, E)$ reprezentowanego przez macierz sąsiedztwa A , algorytm ZNAJDŹCYKLE poprawnie identyfikuje wszystkie cykle, przy czym cykle są definiowane jako zamknięte ścieżki w grafie, w których każdy wierzchołek poza punktem początkowym jest odwiedzany najwyżej raz.

Podstawa indukcji: Rozważmy graf o $n = 1$ wierzchołku. Macierz sąsiedztwa A ma wymiar 1×1 , a jedyną potencjalną krawędzią jest pętla $A[1][1]$, która tworzy cykl o długości 1.

Algorytm: 1. Inicjalizuje $\text{cycle_count} \leftarrow 0$. 2. Rozpoczyna przeszukiwanie w głąb (DFS) od wierzchołka $u = 1$. 3. Sprawdza, czy długość aktualnej ścieżki path wynosi co najmniej 3 (co jest niemożliwe dla $n = 1$). 4. Kończy działanie, zwracając $\text{cycle_count} = 0$, co jest poprawne.

Podstawa indukcji została wykazana, ponieważ graf o jednym wierzchołku nie zawiera cykli o długości większej niż 2.

Krok indukcyjny: Załóżmy, że algorytm poprawnie identyfikuje wszystkie cykle w dowolnym grafie $G_k = (V_k, E_k)$ o $n = k$ wierzchołkach. Udowodnijmy, że działa poprawnie dla grafu $G_{k+1} = (V_{k+1}, E_{k+1})$ o $n = k + 1$ wierzchołkach.

Założenie indukcyjne: Dla dowolnego grafu G_k , algorytm identyfikuje wszystkie cykle $C \subseteq G_k$, przy czym każdy cykl spełnia warunek:

$$C = \{v_1, v_2, \dots, v_m\}, \quad v_1 = v_m, \quad v_i \neq v_j \text{ dla } i \neq j.$$

Dowód dla $n = k + 1$: Rozważmy graf $G_{k+1} = (V_{k+1}, E_{k+1})$ o $k + 1$ wierzchołkach. Algorytm rozpoczyna przeszukiwanie w głąb (DFS) od każdego wierzchołka u i wykonuje następujące kroki: 1. Dodaje u do aktualnej ścieżki path i oznacza jako odwiedzony w zbiorze visited . 2. Rekurencyjnie eksploruje wszystkich sąsiadów v , którzy nie należą do visited . 3. Jeżeli wierzchołek startowy u zostanie osiągnięty po co najmniej dwóch krokach ($\text{length}(\text{path}) > 2$), algorytm zlicza cykl.

Każdy potencjalny cykl w G_{k+1} należy do jednej z dwóch kategorii: - $C \subseteq G_k$: Cykl istnieje wyłącznie w podgrafie G_k , a poprawność jego detekcji wynika z założenia indukcyjnego. - $C \not\subseteq G_k$: Cykl obejmuje nowy wierzchołek v_{k+1} .

Dla $C \not\subseteq G_k$, algorytm identyfikuje cykl C w sposób następujący: - Przeszukiwanie w głąb obejmuje wszystkie sąsiadujące wierzchołki $v \in V_k$, które mają krawędź do v_{k+1} (czyli $A[v][v_{k+1}] > 0$). - Warunek $u = \text{start}$ po powrocie do wierzchołka początkowego u gwarantuje, że cykl został zamknięty.

Złożoność rekurencji wynosi $O(k + 1 + E_{k+1})$ dla każdego wierzchołka u , co zapewnia, że wszystkie krawędzie są odwiedzone dokładnie raz.

Wniosek: Na mocy zasady indukcji matematycznej algorytm ZNAJDŹCYKLE poprawnie identyfikuje wszystkie cykle w grafie o dowolnej liczbie wierzchołków n .

4.3 Dowód poprawności algorytmu liczenia cykli Hamiltona

Udowodnijmy poprawność algorytmu POLICZCYKLEHAMILTONA (Algorytm 3), który wykorzystuje technikę rekurencyjnego backtrackingu, aby znaleźć wszystkie cykle Hamiltona w grafie $G = (V, E)$.

Teza: Algorytm POLICZCYKLEHAMILTONA poprawnie identyfikuje wszystkie cykle Hamiltona w grafie G , czyli wszystkie permutacje wierzchołków $\pi \in V$ spełniające warunek:

$$\forall i (1 \leq i < n) : (v_i, v_{i+1}) \in E \text{ oraz } (v_n, v_1) \in E.$$

Podstawa indukcji: Rozważmy graf G_1 o $n = 1$ wierzchołku. Macierz sąsiedztwa A jest macierzą 1×1 , a jedyną potencjalną krawędzią jest pętla $A[1][1]$. Algorytm: 1. Inicjalizuje $\text{count} \leftarrow 0$. 2. Rozpoczyna przeszukiwanie w głąb (DFS) od $u = 1$. 3. Osiąga głębokość $\text{depth} = |V| = 1$, sprawdza $A[1][1]$ i jeśli $A[1][1] > 0$, zlicza cykl Hamiltona.

Dla G_1 , istnieje dokładnie jeden cykl Hamiltona (pętla wierzchołka na sobie), jeśli $A[1][1] > 0$. Algorytm zwraca poprawny wynik, $\text{count} = 1$, lub $\text{count} = 0$ w przypadku $A[1][1] = 0$.

Krok indukcyjny: Załóżmy, że algorytm działa poprawnie dla grafu $G_k = (V_k, E_k)$ o $n = k$ wierzchołkach. Udowodnijmy, że działa poprawnie dla $G_{k+1} = (V_{k+1}, E_{k+1})$.

Założenie indukcyjne: Algorytm identyfikuje wszystkie cykle Hamiltona w G_k , tj. każdą permutację π wierzchołków w V_k , spełniającą warunek:

$$\forall i (1 \leq i < k) : (v_i, v_{i+1}) \in E_k \text{ oraz } (v_k, v_1) \in E_k.$$

Dowód dla G_{k+1} : Rozważmy graf G_{k+1} , gdzie dodano wierzchołek v_{k+1} oraz potencjalne krawędzie (v_{k+1}, v_i) dla $v_i \in V_k$. Algorytm: 1. Rozpoczyna DFS od każdego $u \in V_{k+1}$. 2. Dodaje u do zbioru visited i ścieżki path. 3. Rekurencyjnie eksploruje sąsiadów v , które nie znajdują się w visited. 4. W głębokości $\text{depth} = |V| = k + 1$ sprawdza, czy istnieje krawędź powrotna $A[v_{k+1}][v_1]$.

Analiza poprawności: 1. Dla każdego $v \in V_{k+1}$, rekurencja odwiedza dokładnie $k + 1$ wierzchołków w grafie. 2. Warunek zakończenia rekurencji ($\text{depth} = |V|$ i $A[u][\text{start}] > 0$) gwarantuje, że algorytm rozpoznaje tylko cykle Hamiltona. 3. Wszystkie potencjalne permutacje odwiedzenia wierzchołków π są eksplorowane dzięki iteracji $\forall v \in V_{k+1}$ oraz śledzeniu odwiedzonych wierzchołków w visited.

Redukcja do przestrzeni języków formalnych: Cykl Hamiltona można opisać w języku formalnym L_H :

$$L_H = \{\pi \in \Sigma^n : \forall i (1 \leq i < n) (v_i, v_{i+1}) \in E \text{ oraz } (v_n, v_1) \in E\}.$$

Algorytm można zredukować do maszyny Turinga M_H , która generuje wszystkie permutacje $\pi \in \Sigma^n$ i akceptuje je, jeśli $\pi \in L_H$. Maszyna M_H działa w czasie $O(n!)$ na wejściu długości n .

Złożoność: Dla każdego podzbioru odwiedzonych wierzchołków $S \subseteq V$, DFS wykonuje $O(n)$ kroków w celu eksploracji sąsiadów i $O(n^2)$ operacji w sprawdzaniu warunku zakończenia rekurencji. Łączna liczba podzbiorów wynosi 2^n , co prowadzi do złożoności $O(2^n \cdot n^2)$.

Wniosek: Na mocy indukcji matematycznej oraz zgodności z językiem L_H , algorytm POLICZCYKLEHAMILTONA poprawnie identyfikuje wszystkie cykle Hamiltona w grafie G .

4.4 Dowód poprawności algorytmu obliczania metryki pomiędzy dwoma multigrafami

Algorytm obliczania metryki pomiędzy dwoma multigrafami (OBLICZMETRYKĘ, Algorytm 5) opiera się na dwóch kluczowych krokach: 1. Obliczeniu i normalizacji rozkładu stopni wierzchołków. 2. Zastosowaniu metryki Earth Mover's Distance (EMD) do porównania znormalizowanych rozkładów.

Przeprowadzimy formalny dowód poprawności algorytmu, wykorzystując indukcję matematyczną, teorię maszyn Turinga oraz przestrzeni języków formalnych.

Teza: Algorytm OBLICZMETRYKĘ poprawnie oblicza metrykę EMD między dwoma grafami $G = (V_G, E_G)$ i $H = (V_H, E_H)$, spełniając:

$$\text{metric}(G, H) = \sum_{d \in D} |F_G(d) - F_H(d)|,$$

gdzie $F_G(d)$ oraz $F_H(d)$ są skumulowanymi rozkładami stopni dla G i H , a D to zbiór możliwych stopni.

Dowód krokowy: 1. Obliczenie rozkładu stopni: Dla każdego wierzchołka $v \in V$, algorytm oblicza stopień $\deg(v) = \sum_{j=1}^n A[v][j]$. Rozkład $\text{dist}(d)$ przypisuje liczbie d licznosc wierzchołków $|V_d|$ o tym stopniu:

$$\text{dist}(d) = |\{v \in V : \deg(v) = d\}|.$$

Złożoność czasowa wynosi $O(n^2)$, ponieważ algorytm iteruje przez macierz sąsiedztwa A dla n wierzchołków. Normalizacja rozkładu dzieli licznosc każdego stopnia d przez n , co zapewnia, że:

$$\sum_d \text{dist}(d) = 1.$$

2. Zastosowanie metryki EMD: EMD oblicza minimalny koszt transportu między dwoma znormalizowanymi rozkładami dist_1 i dist_2 . Skumulowany rozkład $F(d)$ jest zdefiniowany jako:

$$F(d) = \sum_{k \leq d} \text{dist}(k),$$

a metryka jest dana przez:

$$\text{metric} = \sum_{d \in D} |F_G(d) - F_H(d)|.$$

Algorytm iteruje po D , wykonując $O(D)$ operacji.

3. Dowód poprawności przez indukcję:

Podstawa indukcji: Dla G i H składających się z jednego wierzchołka ($n = 1$), macierz sąsiedztwa to $A_G[1][1]$ i $A_H[1][1]$. Rozkład stopni wynosi:

$$\text{dist}_G(d) = \begin{cases} 1 & \text{jeśli } d = A_G[1][1], \\ 0 & \text{w przeciwnym razie.} \end{cases}$$

Podobnie dla H . Metryka EMD wynosi $|F_G(d) - F_H(d)| = |A_G[1][1] - A_H[1][1]|$, co algorytm poprawnie oblicza w $O(1)$.

Krok indukcyjny: Załóżmy, że algorytm działa poprawnie dla grafów $G_k = (V_k, E_k)$ i $H_k = (V_k, E_k)$, gdzie $|V_k| = k$. Pokażemy poprawność dla G_{k+1} i H_{k+1} o $|V| = k + 1$.

Dodanie wierzchołka v_{k+1} do G_k oraz H_k wpływa na rozkład $\text{dist}(d)$ przez:

$$\text{dist}'_G(d) = \text{dist}_G(d) + \delta(d, \deg(v_{k+1})),$$

gdzie $\delta(x, y) = 1$ jeśli $x = y$, w przeciwnym razie 0. Skumulowany rozkład $F(d)$ zmienia się zgodnie z definicją, a algorytm poprawnie uwzględnia nowe wierzchołki, aktualizując skumulowane różnice $F_G(d) - F_H(d)$ w $O(D)$.

4. Redukcja do maszyny Turinga: Rozkład stopni wierzchołków można opisać w języku formalnym:

$$L_{\text{dist}} = \{\text{dist}(d) \mid \text{dist}(d) \text{ jest znormalizowanym rozkładem stopni}\}.$$

Maszyna Turinga M_{dist} generuje rozkład $\text{dist}(d)$ w czasie $O(n^2)$ oraz iteruje po stopniach D , obliczając metric w czasie $O(D)$.

5. Złożoność: Łączna złożoność algorytmu wynosi $O(n^2 + D)$, co jest ograniczone przez liczby wierzchołków n i maksymalny stopień D .

Wniosek: Na mocy indukcji matematycznej oraz definicji maszyn Turinga, algorytm OBLICZMETRYKĘ poprawnie oblicza metrykę EMD między dwoma grafami G i H .

4.5 Dowód poprawności algorytmu minimalnego rozszerzenia dla cyklu Hamiltona

Algorytm minimalnego rozszerzenia dla cyklu Hamiltona (MINIMALNEROZSZERZENIE, Algorytm 5) znajduje minimalną liczbę krawędzi, które należy dodać do grafu G , aby uzyskać graf zawierający cykl Hamiltona. Dowód poprawności algorytmu opiera się na kilku elementach teorii grafów, przestżeniu języków formalnych oraz maszyn Turinga. Zastosujemy także indukcję matematyczną i analizę przestrzeni poszukiwań.

Teza: Algorytm MINIMALNEROZSZERZENIE poprawnie oblicza minimalną liczbę krawędzi k , które należy dodać do grafu $G = (V, E)$, aby uzyskać cykl Hamiltona.

Dowód: 1. Konstrukcja algorytmu:

Algorytm działa w sposób rekurencyjny. Dla każdego grafu G' , uzyskanego przez dodanie k krawędzi do G , algorytm: 1. Sprawdza, czy G' zawiera cykl Hamiltona, korzystając z funkcji pomocniczej HASHAMILTONIANCYCLE. 2. Jeśli G' zawiera cykl Hamiltona, zapisuje k jako kandydat na minimalną wartość i kończy dalsze eksplorowanie dla tej ścieżki. 3. W przeciwnym razie rekurencyjnie generuje wszystkie możliwe rozszerzenia G' przez dodanie kolejnych krawędzi.

2. Redukcja do przestrzeni języków:

Każdy graf G można opisać w języku formalnym jako zbiór jego krawędzi:

$$L_G = \{(u, v) \mid (u, v) \in E\}.$$

Dodanie k krawędzi do G można zinterpretować jako generowanie nowego języka $L_{G'}$:

$$L_{G'} = L_G \cup \{(u, v) \mid (u, v) \notin E, |L_{G'}| = |L_G| + k\}.$$

Język $L_{\text{Hamiltonian}}$ zawiera wszystkie grafy z cyklem Hamiltona:

$$L_{\text{Hamiltonian}} = \{G' \mid G' \text{ zawiera cykl Hamiltona}\}.$$

Zadaniem algorytmu jest znalezienie najmniejszego k , dla którego $L_{G'} \cap L_{\text{Hamiltonian}} \neq \emptyset$.

3. Dowód poprawności przez indukcję:

Podstawa indukcji: Dla grafu G o $|V| = 1$, cykl Hamiltona istnieje wtedy, gdy nie ma potrzeby dodawania krawędzi ($k = 0$). Algorytm poprawnie zwraca $k = 0$, ponieważ HASHAMILTONIANCYCLE(G) jest prawdziwe dla grafu o pojedynczym wierzchołku.

Krok indukcyjny: Załóżmy, że algorytm działa poprawnie dla grafów o $|V| = n$. Rozważmy graf G o $|V| = n + 1$.

Dodanie krawędzi do G jest równoważne rozszerzeniu języka L_G . Algorytm generuje wszystkie $\binom{n+1}{2} - |E|$ możliwe krawędzie w sposób rekurencyjny. Dla każdej generowanej krawędzi: 1. Funkcja HASHAMILTONIANCYCLE sprawdza, czy nowy graf G' zawiera cykl Hamiltona. 2. Jeśli tak, algorytm aktualizuje minimalną liczbę dodanych krawędzi k i kończy dalsze rekurencje dla tego rozszerzenia.

Indukcyjnie, dla każdego k , algorytm eksploruje wszystkie możliwe rozszerzenia G' , co zapewnia, że minimalny k zostanie znaleziony.

4. Analiza na maszynie Turinga:

Operację dodania krawędzi do grafu G można zrealizować na maszynie Turinga M , która iteruje przez przestrzeń 2^m , gdzie m to liczba brakujących krawędzi w grafie: 1. Maszyna M generuje każdą możliwą kombinację k dodanych krawędzi. 2. Dla każdego grafu G' , podjęta jest decyzja, czy G' zawiera cykl Hamiltona, co jest decyzyjne w czasie $O(n!)$ (lub $O(2^n)$ dla zoptymalizowanych algorytmów). 3. Wynik jest minimalnym k , dla którego G' spełnia warunek cyklu Hamiltona.

5. Złożoność:

Całkowita liczba konfiguracji wynosi $O(2^m)$, a dla każdego grafu sprawdzenie cyklu Hamiltona zajmuje $O(n!)$. Łączna złożoność wynosi:

$$O(2^m \cdot n!),$$

gdzie $m = \binom{n}{2} - |E|$.

Wniosek: Na mocy indukcji matematycznej oraz redukcji do języków formalnych, algorytm MINIMALNEROZSZERZENIE poprawnie oblicza minimalną liczbę krawędzi k , które należy dodać, aby uzyskać cykl Hamiltona w G .

5 Instrukcja obsługi

5.1 Struktura projektu

Projekt składa się z kilku kluczowych katalogów i plików:

- **src/** - Katalog zawierający kody źródłowe programu głównego.
- **data/** - Przykładowe pliki wejściowe z danymi grafów.
- **results/** - Katalog, w którym zapisywane są wyniki analizy grafów.
- **documentation.pdf** - Plik dokumentacji projektu, w tym opis algorytmów, dowody i przykłady użycia.
- **graph_gen.exe** - Program generujący przykładowe grafy o różnych strukturach.
- **Makefile** - Plik automatyzujący kompilację projektu.
- **test/** - Testy projektu wraz ze skrypcem **test_runner.sh** do uruchamiania testów automatycznych.

5.2 Kompilacja projektu

Do kompilacji programu i generatora grafów należy użyć polecenia:

```
make all
```

Po zakończeniu kompilacji zostaną utworzone dwa pliki wykonywalne:

- **multigraph_analysis.exe** - Program główny do analizy grafów.
- **graph_gen.exe** - Program do generowania przykładowych grafów.

5.3 Uruchamianie programu głównego

Program główny **multigraph_analysis.exe** wymaga pliku wejściowego z opisem grafów. Plik wejściowy powinien mieć format:

```
<number_of_graphs>
<number_of_vertices_for_graph_1>
<adjacency_matrix_for_graph_1>
<number_of_vertices_for_graph_2>
<adjacency_matrix_for_graph_2>
...
```

Przykład pliku wejściowego:

```
2
3
0 1 1
1 0 1
1 1 0

4
0 2 0 1
2 0 1 0
0 1 0 1
1 0 1 0
```

Uruchomienie programu:

```
./multigraph_analysis.exe <input_file>
```

Przykład:

```
./multigraph_analysis.exe data/input.txt
```

Program przyjmuje jedynie jeden argument, jednak obsługuje wiele grafów w jednym pliku wejściowym. W celu obliczenia metryki pomiędzy grafami, należy umieścić je w jednym pliku, jeden pod drugim. Wyniki analizy grafów zostaną wyświetlone w terminalu i mogą być przekierowane do pliku wyjściowego.

5.4 Opis funkcji programu głównego

- `handle_arguments(int argc, char *argv[], Config *config)` - Przetwarza argumenty wiersza poleceń. Oczekuje podania pliku wejściowego.
- `open_file_with_retry(const char *filename)` - Otwiera plik wejściowy z danymi grafów, obsługując ponowne próby w przypadku zakłóceń systemowych.
- `print_cycles(GArray *output_cycles)` - Wyświetla cykle znalezione w grafie.
- `analyze_multigraph(GraphInterface *multigraph)` - Wykonuje analizę pojedynczego grafu, w tym:
 - Liczenie rozmiaru grafu.
 - Znajdowanie wszystkich cykli.
 - Znajdowanie cykli Hamiltona.
 - Obliczanie minimalnego rozszerzenia dla cyklu Hamiltona.

5.5 Uruchamianie generatora grafów

Program `graph_gen.exe` pozwala generować grafy o różnej strukturze. Wywołanie:

```
./graph_gen.exe <graph_type> <num_graphs> <vertices> <max_weight> <output_file>  
<extra_parameter>
```

- `<graph_type>` - Typ grafu:
 - `complete` - Pełny graf skierowany.
 - `sparse` - Rzadki graf skierowany.
 - `bipartite` - Dwudzielny graf skierowany.
- `<num_graphs>` - Liczba grafów do wygenerowania.
- `<vertices>` - Liczba wierzchołków.
- `<max_weight>` - Maksymalna waga krawędzi.
- `<output_file>` - Nazwa pliku wyjściowego.
- `<extra_parameter>` - Dodatkowy parametr:
 - Dla `sparse`: Prawdopodobieństwo krawędzi ($0.0 < p \leq 1.0$).
 - Dla `complete` i `bipartite`: Maksymalna liczba krawędzi między parami wierzchołków.

5.6 Przykłady użycia generatora grafów

Generowanie pełnego grafu:

```
./graph_gen.exe complete 1 5 10 data/complete_graph.txt 3
```

Generowanie rzadkiego grafu:

```
./graph_gen.exe sparse 1 5 10 data/sparse_graph.txt 0.2
```

Generowanie grafu dwudzielnego:

```
./graph_gen.exe bipartite 1 10 10 data/bipartite_graph.txt 3
```

5.7 Uruchamianie testów

Testy można uruchomić za pomocą skryptu `test_runner.sh`:

```
bash test/test_runner.sh
```

Skrypt wykonuje analizę predefiniowanych przypadków testowych oraz generuje dodatkowe testy za pomocą programu `graph_gen`. Wyniki testów są zapisywane w katalogu `results/`.

6 Wnioski

- **Złożoność obliczeniowa algorytmów:** zaimplementowane algorytmy prezentują różne rzędy złożoności obliczeniowej. Najbardziej obciążające są algorytmy dotyczące cykli Hamiltona, które mają złożoność rzędu silni. Takie algorytmy mogą sobie nie radzić z dużymi grafami, dlatego zaleca się stosowanie algorytmów aproksymacyjnych. Z tego powodu właśnie zdecydowaliśmy się na wybranie takiej, a nie innej metryki. Dzięki złożoności kwadratowej można analizować duże grafy w dość krótkim czasie, nie tracąc dużo na przydatności wyniku.
- **Zastosowanie w praktyce:** wiele problemów praktycznych można sprowadzić do postaci analizy grafu. Korzystając z zaimplementowanych algorytmów, możemy rozwiązywać problemy optymalizacyjne oraz analizować struktury grafów.